

BUILD MANUAL

RaceChrono DIY CAN-Bus アダプタ 製作手順書

ESP32 + SN65HVD230 / Abarth 595 シリーズ4 向け

対象車両	Abarth 595 シリーズ4 (Fiat 500 / 312系)
マイコン	Freenove ESP32-WROOM ボード (FNK0090B)
ファームウェア	katsuhaa / racechrono-diy-esp32 ブランチ <code>abarth595-esp32-wroom</code> (kevenduchesneau 上流 <code>c2290a9</code> に <code>config.h</code> ・本書・ <code>tools/</code> を追加)
動作モード	受信専用 (Listen Only。トランシーバ D は「5」に接続、常時レセッシブ)
版数	Rev. 1.8 (実車で CAN ID を確定、D の配線を変更、受信確認と ID 特定を <code>can_monitor</code> に置換)
作成日	2026年9月7日
ビルド確認	arduino-esp32 3.3.11 / NimBLE-Arduino 1.4.3 / EasyLogger 1.1.4 / ESP32 Dev Module (604,612 bytes)

1. 本書の概要と設計方針

本書は、Abarth 595 シリーズ4の OBD-II ポートから CAN バスのデータを読み取り、Bluetooth LE 経由でスマートフォンアプリ「RaceChrono Pro」へ送信する DIY データロガーの製作手順書です。Rev.1.0 の技術レビューと、その後の設計簡素化（保護部品・抵抗・電源スイッチ・外付けコンデンサの削除）、配線のシルク表記への統一を経て、Rev.1.8 では実車で受信確認と CAN ID の特定結果を反映しました。回転数・アクセル・舵角・車速・4 輪速・ブレーキ・クラッチの ID とバイト位置は第3章、RaceChrono の設定値は 10-3 にあります。あわせて、トランシーバの D（TXD）を「開放」から Freenove の「5」への接続に変更しました（理由は第5章のコラム）。変更の経緯は末尾の改訂履歴を参照してください。

設計上の4つの原則

原則	内容
完全受信専用	車両の CAN バスへ一切の信号を送出しない。ファームウェアの TWAI を Listen Only モードで動かし、ACK もエラーフレームも出さない。トランシーバの D は「5」（TX_PIN）に接続するが、Listen Only では TX が L になることは無く、常にリセット。
バス負荷ゼロ	トランシーバ基板上の終端抵抗を除去し、車両側バスのインピーダンスに影響を与えない。
電源の切り離し	OBD-II の 12V は常時通電のため、使わないときは装置を OBD ポートから物理的に抜く。電源スイッチは設けない。
段階的に確認	机上（USB 給電）→ 机上（12V 給電）→ 車両（USB 給電・12V 線は未接続）→ 車両（12V 運用）の順に、1 段階ずつ動作を確認してから次へ進む。確認はすべて装置自身が表示する結果（LED・シリアル）で行い、テスターの直流値からの推測で判断しない。

重要：車両側の安全に関する前提

本装置は走行中の車両の通信バスに接続します。誤配線や設定ミスはパワーステアリング・ABS・エンジン制御に影響を及ぼす可能性があります。第9章のベンチテストを完了するまで、車両に接続しないでください。また、公道での初回接続は避け、必ず安全な場所での停車状態から確認を始めてください。

2. 部品表（BOM）

Rev.1.0 から削除した部品：ヒューズ、逆接続保護ダイオード、TVS、10kΩ 抵抗、電源スイッチ、電解コンデンサ×2（経緯は改訂履歴）。残りは 7 品目。

部品	仕様・型番	役割と備考
マイクロコントローラ	Freenove ESP32-WROOM 開発ボード（FNK0090B）	メイン処理・BLE通信。ESP32-WROOM-32E 搭載、デュアルコア240MHz / 4MBフラッシュ / BT 4.2 LE、USB-シリアルは CH340。ピンヘッダは実装済みのため、薄型化のため除去を推奨（第4章）。ピン名はシルクに数字だけで印刷されている（第5章）。

CANトランシーバ	SN65HVD230 ブレイクアウト基板（VP230 刻印の青い基板）	CAN差動信号を3.3Vロジックへ変換。基板上の120Ω終端抵抗は必ず除去すること（第4章）。この基板のICはD（TXD）を開放にするとRが出なくなるので、Dは必ずHに固定する（第5章）。Vref ピンが0Vでも正常（第11章）。
OBD-IIコネクタ	SAE J1962 Type A 16ピン オス（金属ラッチ・圧着端子）	車両ポートへの接続。金属ラッチにより強いGでの脱落を防止。付属オープンバレル端子はAWG24〜20 対応。
降圧モジュール	MP1584EN 可変DC-DCモジュール	車両12Vを5Vへ降圧。接続前に必ず5.0Vへ調整（第6章）。
配線材	AWG24 耐熱シリコンワイヤー	モジュール間結線。IWISS SN-QC-WC で圧着可能、狭い筐体でも取り回し良好。
絶縁・固定材	カプトンテープ、熱収縮チューブ、結束バンド	ショート防止と、筐体・車両ハーネスの物理固定。
筐体	3Dプリント品（タイラップ用スリット付き）	OBDポート周辺は膝が当たるため、可能な限り薄型に設計する。夏季の車内温度を考慮し PLA は避ける。

補足：予備のボードとモジュールについて

Freenove のボードは2個パックで購入済みのため、1枚はベンチテスト・練習用として温存することを推奨します。SN65HVD230 基板も 2 枚あると、受信部が動かなくなったときに載せ替えて切り分けができます（製作中に 1 枚が受信不能になり、もう 1 枚を本番に使った経緯があります。改訂履歴 Rev.1.8）。

3. 対象車両のCANバス仕様と確認済み CAN ID

Abarth 595 シリーズ4は Fiat 500（312系）プラットフォームを共有しており、高速側と低速側の2系統のCANバスを持ちます。実車で確認した結果、C-CAN（OBD ピン 6/14、500 kbps）には常時 27 個の ID が流れ、すべて 29 ビット（拡張）ID でした。本書で必要な信号は舵角も含めてすべて C-CAN 側にあり、CAN-B 用の 2 台目は不要です。ファームウェアは全 ID 受け入れ・500 kbps 固定なので設定変更は不要です。

バス	速度	OBDコネクタ	主に流れるデータ
CAN-C（高速）	500 kbps	ピン 6 / 14	エンジン制御、ABS/ESC、EPS。回転数・車速・4輪速・アクセル・舵角・ブレーキ・クラッチ（下表）
CAN-B（低速）	50 kbps	ピン 1 / 9	ボディ系、メーター、快適装備。ウインカーは C-CAN に出ないのでこちら側と推定。本書では使用しない

3-1. 確認済み CAN ID（2026-09-07、停車・徐行の実車ログ）

第10章の手順（tools/can_monitor）で、操作と同時に変化したバイトから特定しました。10 進の ID は RaceChrono の PID 欄に入力する値です。バイト番号は 0 始まり、BE はビッグエンディアン（上位バイトが先）。

信号	ID（16進 / 10進）	バイトと式	確認内容	確度
エンジン回転数	0x0618A001 102277121	b2-3 BE = rpm	アイドル 790、空吹かし 1491、エンジン停止で 0	確定
アクセルペダル	0x0618A001 102277121	b7 / 2.55 = %（0～255）	IG ON・エンジン停止で 1/4・1/2・全開の保持が 0x1F・0x86・0xFF	確定
舵角	0x0030A002 3186690	(b3b4b5 >> 4) の 20 ビット。1 count = 0.1°、左が増加。中立 ≈ 7400	左 90° で +820、右 90° で -1100、ロック to ロックで 2615～11740（912° ≈ 2.5 回転）。b6 は操舵速度相当、b7 はカウンタ/チェックサム	確定 （中立値は要微調整）
車速	0x0210A006 34643974	b4-5 BE / 128 = km/h	徐行で 4.8 → 9.7 → 13.8 km/h	確定
4 輪速	0x0218A006 35168262	b0-1, b2-3, b4-5, b6-7 の各 BE / 16 = km/h	13.8 km/h 時に 221・216・223・218（左前・右前・左後・右後の順と推定、未確認）	確定 （順序は五分）
ブレーキスイッチ	0x0810A000 135307264	b2 の上位ニブル：1 = 離し、3 = 軽く、7 = 踏み	踏むと 0x10 → 0x70	確定
ブレーキ圧	0x0010A006 1089542	b1-2 BE（0～約 0x11B、単位未確定）	踏力に応じて連続的に変化、離すと 0	有力
クラッチ	0x0628A001 103325697	b5 の bit5（0x20）= 踏み	踏むと 0x00 → 0x20（アクセル ON で bit4 0x10 も立つ）	有力

参考：まだ確定していない信号

0x0220A006 b0-1（0x8000 中心、加減速で変化）と 0x0028A00F b6（制動で +、加速で -）は前後 G の候補（五分）。0x0028A00F の b3 は操舵中に動くのでヨーレート系の候補（五分）。RaceChrono は G を端末または RaceBox から取るため、本書では使いません。0x0628A001 b4（アクセルに非線形で追従）と 0x0618A001 b4（スロットル開度相当）はペダルの代わりには使いません。ウインカー・ライトは C-CAN に出ません。

出典（Rev.1.5～1.7 の候補の根拠、履歴として残す）：github.com/P1kachu/talking-with-cars（notes/fiat-500.txt）、github.com/VeryBusyBee/FiatMon、Racelogic「Fiat 500 2007-」、AiM「Fiat 500 Abarth (from 2008 onward)」。候補のうち回転数・車速・4輪速・ブレーキ・クラッチの ID は実車で一致しました。

4. 事前加工（組み立て前に必ず実施）

4-1. SN65HVD230 基板の120Ω終端抵抗を除去する

市販の SN65HVD230 ブレイクアウト基板の多くには、CANH-CANL 間に 120Ω の終端抵抗が実装されています。車両の CAN バスは既に両端が 120Ω で終端されているため、これを追加すると3本並列となりバスインピーダンスが約 40Ω まで低下し、通信エラーの原因となります。

1. 基板上で CANH と CANL の端子間に配置された抵抗（表面実装、刻印「121」または「1200」。青い基板では R2）を特定する。
2. はんだごてで両端を温めながらピンセットで除去する。ジャンパ形式の基板であればカットする。
3. テスターで CANH-CANL 間の抵抗値を測定し、120Ω 付近でないことを確認する。トランシーバ内部の入力抵抗により数十 kΩ 程度が読める、または OL 表示になる。どちらも正常。

4-2. Rs（スロープ制御）ピンの確認

Rs ピンがジャンパで切り替え可能な基板の場合は、高速モード（Rs をグランド側）に設定します。抵抗で固定されている基板はそのまま問題ありません。Rs は主に送信側のスルーレートを決めるピンで、GND 側（高速）または抵抗で GND に落とした状態（スロープ制御）であれば受信には影響しません。手持ちの青い基板では Rs（8 番ピン）は 0V でした。

4-3. ESP32 ボードのピンヘッダ除去（任意）

Freenove のボードはピンヘッダがはんだ付け済みで出荷されます（2列 × 20 = 40ピン）。一括で吸い取るとランドが剥がれるため、以下の順序で作業してください。

1. ピンヘッダの黒い樹脂部分をニッパーで破壊し、除去する。
2. 残った金属ピンを1本ずつラジオペンチで掴み、はんだごてで温めながら引き抜く。
3. すべて抜き終わったら、はんだ吸取り線でスルーホールを整える。

※ 薄型化が不要であれば、この工程は省略しても機能上の問題はありません。ピン名のシルクは各スルーホールの脇に印刷されているので、ヘッダを抜いても読めます。

5. 結線

本章のピン名は、すべて基板に印刷されている文字（シルク）で表記します。Freenove ボードでは GPIO 番号が数字だけで印刷されているため、config.h の RX_PIN 4 はシルクの「4」、TX_PIN 5 は「5」に対応します。電源ピンのシルクは「3.3V」「5V」「GND」です（「3V3」「VIN」という表記はこのボードにはありません）。

5-1. Freenove ボードで使うピン

アンテナ側（モジュールの金属缶がある側）を上、USB-C コネクタを下にして見ます。使うのは右列の「4」と「5」、左列の「3.3V」1本と「5V」1本、右列の「GND」2本の計6本です。

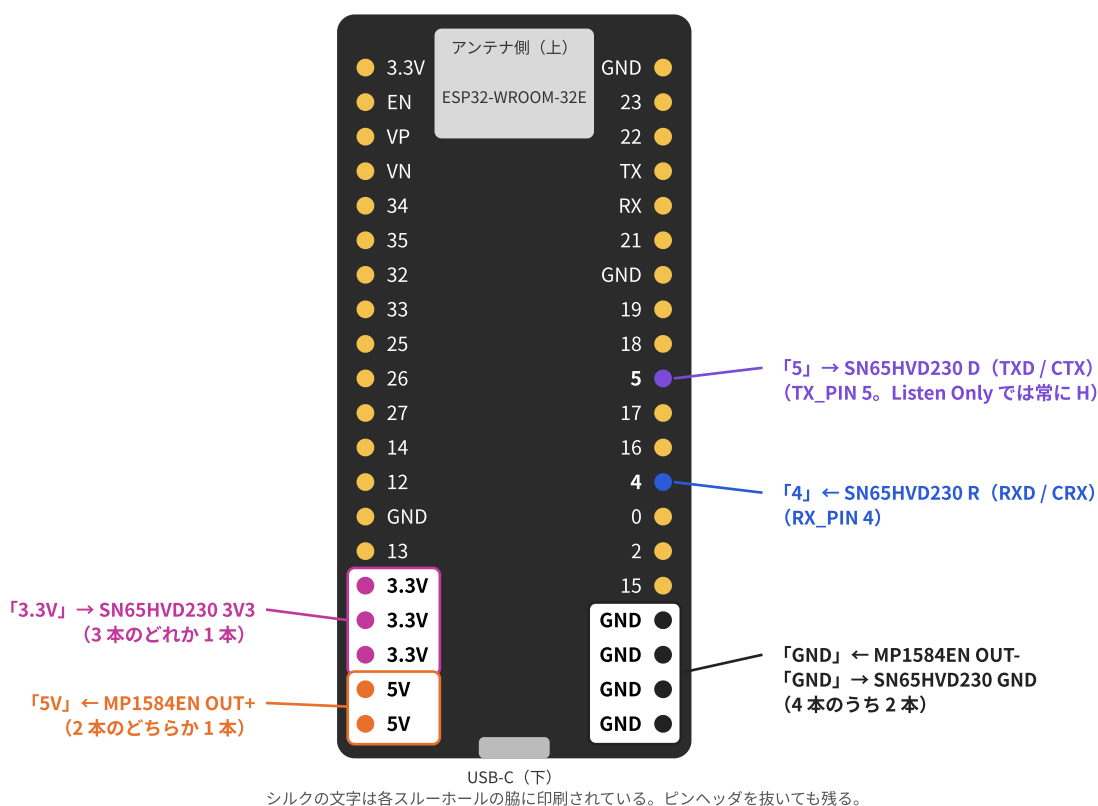


図 5-1 Freenove ESP32-WROOM ボード (FNK0090B) のシルク表記と使用ピン。配置は Freenove 公式のピン配置図 (GitHub リポジトリ Freenove_ESP32_WROOM_Board) に基づく。

シルク	場所	用途
4	右列、上から 13 番目。「16」と「0」の間	CAN 受信入力 (config.h の RX_PIN 4)。 SN65HVD230 の R へ
5	右列、上から 10 番目。「18」と「17」の間	CAN 送信出力 (config.h の TX_PIN 5)。 SN65HVD230 の D へ。Listen Only では常に H (レセ ツシブ) で、D をプルアップする役割だけを持つ
3.3V	左列の下側、白枠内の 3 本 (左列の最上段 も 3.3V)。どれか 1 本	SN65HVD230 の電源
5V	左列の最下部、白枠内の 2 本。どれか1本	MP1584EN の 5.0V 出力を入れる
GND	右列の最下部、白枠内の 4 本 (ほかに右列 最上段、「21」の下、左列「12」の下にもあ る)。2 本使う	MP1584EN の OUT- と、SN65HVD230 の GND

5-2. J1962 プラグ (OBD-II コネクタ) で使うピン

端子を入れる位置は **16・5・6・14** の 4 か所 (GND を 2 本にする場合は 4 も) です。ピン番号の並びは見る向きで左右が反転するので、必ず図 5-2 の③ (端子を差し込む側) で数えるか、プラグ本体の刻印番号を見てください。

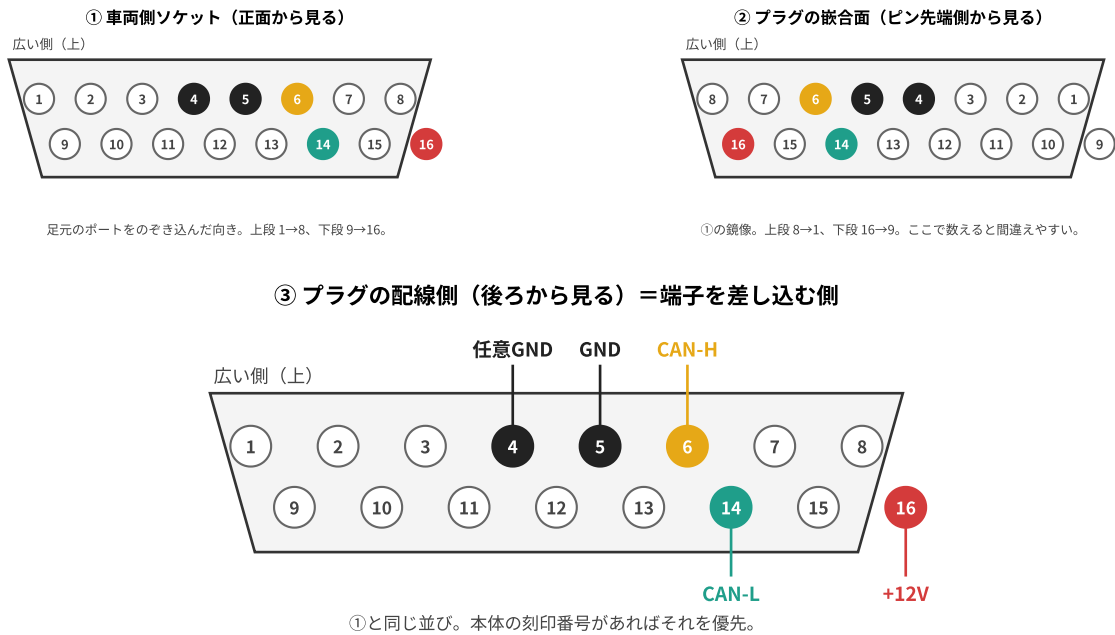


図 5-2 J1962 のピン配置。①車両側ソケットを正面から、②プラグを嵌合面から、③プラグを配線側（後ろ）から見た図。いずれも広い側を上になっている。

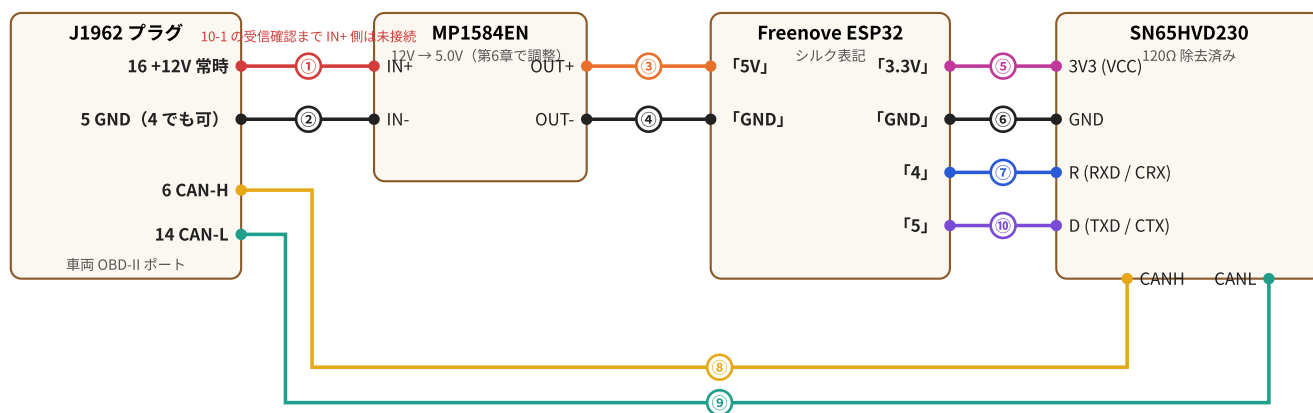
ピン	信号	接続先と備考
16	+12V（バッテリー常時）	MP1584EN の IN+ へ。10-1 の受信確認が終わるまで MP1584EN 側は接続しない（プラグ側の圧着は先にしてよい）
5	GND（信号グラウンド）	MP1584EN の IN- へ。GND はこの 1 本で足りる
4	GND（車体グラウンド）	任意。使う場合はピン 5 の線と一緒に IN- へ
6	CAN-H（CAN-C 高速バス）	SN65HVD230 の CANH へ
14	CAN-L（CAN-C 高速バス）	SN65HVD230 の CANL へ
1・9	CAN-B（低速 50 kbps）	本書では使わない。端子を入れない

※ 付属の端子は AWG24～20 対応で AWG24 には大きめです。被覆を剥いた芯線を折り返して二重にしてから圧着すると抜けにくくなります。圧着後、テスターで 16-5 間および 6-14 間が導通していないことを確認してから車両に挿してください。

5-3. 配線表（1 本ずつ）

全 10 本です。番号は図 5-3 の丸数字に対応します。SN65HVD230 基板と MP1584EN モジュールのピン名は基板の印刷を確認してください（青い SN65HVD230 基板の印刷は 3V3・GND・CTX・CRX・CANH・CANL。CTX が D（TXD）、CRX が R（RXD）です）。

#	起点	終点	信号	備考
①	プラグ 16	MP1584EN IN+	+12V	10-1 の受信確認まで MP1584EN 側を繋がない
②	プラグ 5 (4 でも可)	MP1584EN IN-	GND	
③	MP1584EN OUT+	Freenove 「5V」	5.0V	第6章で 5.00V に調整済みのもの
④	MP1584EN OUT-	Freenove 「GND」	GND	IN- と OUT- はモジュール内部でつながっている
⑤	Freenove 「3.3V」	SN65HVD230 3V3 (VCC)	3.3V	トランシーバ電源。USB 給電のみでも受信動作する (第10章で利用)
⑥	Freenove 「GND」	SN65HVD230 GND	GND	
⑦	Freenove 「4」	SN65HVD230 R (RXD / CRX)	CAN 受信	受信専用
⑧	プラグ 6	SN65HVD230 CANH	CAN-H	6=H、14=L を入れ違えない
⑨	プラグ 14	SN65HVD230 CANL	CAN-L	
⑩	Freenove 「5」	SN65HVD230 D (TXD / CTX)	CAN 送信 (常時 H)	Rev.1.8 で追加。D を開放にしないこと (下のコラム)。「3.3V」に直結してもよい



丸数字は 5-3 の配線表の番号。外付けコンデンサ・スイッチ・抵抗は無し。MP1584EN の IN- と OUT- はモジュール内部でつながっている。

図 5-3 配線図。線の色は 5-3 の表と同じ。

※ 外付けのコンデンサは付けない (MP1584EN モジュールの入出力コンデンサと ESP32 ボード側のコンデンサで足りる。実車で BLE 接続中のリセットは出ていない)。※ 電源スイッチは設けない。※ プラグ 16 → MP1584EN IN+ の 12V 線は 10-1 の受信確認が終わるまで接続しない (USB 給電で作業する)。12V 線を繋いだ後は、USB を挿すときは装置を OBD から抜く。

D (TXD / CTX) を「5」に接続する理由 (Rev.1.8 で変更)

Rev.1.4～1.7 では、TI のデータシート (D は開放でもレセッシブ) を根拠に D を無接続にしていました。しかし実車で 1 フレームも受信できず、机上で切り分けたところ、手持ちの青い VP230 基板では **D を開放にすると R (CRX) が 0V に張り付いて受信できない**ことが分かりました (D に 3.3V を加えると即座に R が H に戻る。同じ症状が 2 枚目の基板でも再現)。互換品の IC には D の内部プルアップが無い、または弱いものがあると考えられます。

そこで D は必ず H に固定します。接続先は Freenove の「5」(config.h の TX_PIN) です。ファームウェアは TWAI を Listen Only モードで起動しており (ソースコードで確認済み)、Espressif の公式ドキュメントでは Listen Only は ACK もエラーフレームも含めドミナントビットを一切送出しません。つまり「5」は常に H で、D をプルアップする役割だけを持ちます。ハードウェア的に送信を完全に封じたい場合は「5」の代わりに「3.3V」へ直結しても動作は同じです。

注意: tools/twai_selftest は「5」を実際に駆動するテストなので、車両に挿した状態では書き込まないでください。

6. 電源電圧の調整

ESP32 を接続する前に必ず実施

MP1584EN の出荷時出力電圧は不定です。調整前の出力が ESP32 に加わると即座に破損します。

調整は自宅で行えます。MP1584EN の入力範囲は 4.5～28V で、出力電圧の設定は入力電圧に依存しない (フィードバックで一定に保たれる) ため、7～28V の DC 電源なら何でも使え、そこで調整した値がそのまま車両の 12～14.5V でも有効です (有力)。手持ちで使えるもの: 9V 乾電池 (006P)、電圧を問わず 7～28V の AC アダプタ (ルーター 12V、モニターや古いノート PC の 19～20V、LED 用 12/24V など)、USB 充電器 2 個の直列 (各出力は絶縁されているので 5V+5V=10V にできる。USB ケーブルを切って赤=+5V・黒=GND を取り出す)、単 3 電池 6 本直列 (アルカリ 9V/ニッケル水素 7.2V。無負荷なのでテープで束ねれば十分)。いずれも無ければ車のシガーソケットの 12V で行う (ESP32 は繋がらない)。

1. MP1584EN の IN+ / IN- に 9V 乾電池 (またはその他 7～28V の DC 電源) を接続する。この時点で ESP32 とトランシーバは接続しない。
2. テスターを OUT+ と OUT- に当て、出力電圧を測定する。
3. 基板上の半固定抵抗 (ポテンショメータ) を精密ドライバーでゆっくり回し、5.00V ± 0.05V に調整する。
4. 入力を切り、再投入して電圧が再現することを確認する。
5. 調整完了後、半固定抵抗にマニキュアまたは接着剤を少量塗布して固定する。車両の振動で設定値がずれるのを防ぐため、この工程は省略しないでください。
6. 電源を切ってから、Freenove の「5V」ピンへ接続する (配線③④)。

7. ソフトウェア環境の構築

7-1. Arduino IDE の準備

- Arduino IDE をインストールし、ボードマネージャで esp32 by Espressif Systems (3.x 系) を導入する。本書は 3.3.11 でビルドを確認済み。作者の開発環境も 3.x 系。
- macOS で書き込みポートが表示されない場合は、CH340 ドライバ (Freenove の GitHub リポジトリに同梱) を導入する。ポート名は `ls /dev/cu.*` で確認する (挿す USB ポートで `cu.usbserial-110` のような番号が変わる)。

7-2. 必要ライブラリ

ライブラリ	導入方法	備考
NimBLE-Arduino	ライブラリマネージャでバージョン 1.4.3 を明示的に選択	BLE スタック。既定でインストールされる 2.x (現行 2.5.1) では <code>arduino-RaceChrono</code> がビルドできない (<code>setScanResponse</code> が廃止、 <code>onWrite</code> の引数が変更)。1.4.3 は 1.x 系の最終版で、esp32 3.x 系に対応。
EasyLogger	ライブラリマネージャ (1.1.4 で確認)	シリアルログ出力
arduino-RaceChrono	手動インストール	ライブラリマネージャには登録されていない。下のコマンドで <code>~/Documents/Arduino/libraries</code> に配置する (macOS の Arduino IDE の既定スケッチブック)。

```
cd ~/Documents/Arduino/libraries
git clone https://github.com/timurrrr/arduino-RaceChrono.git arduino-RaceChrono
```

7-3. ソースコードの取得

1. 自分のフォーク `katsuhaa/racechrono-diy-esp32` のブランチ `abarth595-esp32-wroom` を使う。上流 `kevenduchesneau/racechrono-diy-esp32` (コミット `c2290a9`) に、Abarth 用の `config.h`、本書 (`docs/`)、確認用スケッチ (`tools/`) を追加したもの。手元に無い場合は次のコマンドで取得する。

```
cd ~/Documents/workspace/ODB2CloneForRaceChrono
git clone -b abarth595-esp32-wroom https://github.com/katsuhaa/racechrono-diy-esp32.git
```

2. Arduino IDE で `racechrono-diy-esp32/racechrono-diy-esp32.ino` を開く。同じフォルダの `config.h` がタブとして表示されればよい。ブランチに `config.h` が無い場合は `config.h.example` をコピーして `config.h` とし、7-4 の内容にする (このファイルが無いとビルドできない)。
3. 設定値は 7-4 のとおり。 `getUpdateRateHz()` には第3章で確定した ID が入っている。

※ `.ino` には `arduino-esp32 3.3.x` 向けの 1 行 (`#include <esp32-hal-alloc-ble-mem.h>`) が追加されている。これが無いと起動時に BLE 初期化で `ESP_ERR_INVALID_STATE` が出て再起動を繰り返す (第11章)。

7-4. config.h (全文・リポジトリと同一)

```
#ifndef CONFIG_H
#define CONFIG_H

// Configuration for the Abarth 595 Series 4 (Fiat 500 / 312 platform),
```

```

// classic ESP32-WROOM-32E board (Freenove FNK0090B), receive-only build.
// Abarth 595 シリーズ4 / 無印 ESP32-WROOM (Freenove FNK0090B) 向け・受信専用の設定。

// Bluetooth device name shown in the RaceChrono app
// RaceChrono アプリに表示される Bluetooth デバイス名
#define DEVICE_NAME "RaceChrono DIY ESP32"

// Serial logging level, select one from the following list :
// LOG_LEVEL_EMERGENCY
// LOG_LEVEL_ALERT
// LOG_LEVEL_CRITICAL
// LOG_LEVEL_ERROR
// LOG_LEVEL_WARNING
// LOG_LEVEL_NOTICE
// LOG_LEVEL_INFO
// LOG_LEVEL_DEBUG
// DEBUG prints the TWAI / ring buffer / BLE frame counts every 10 seconds,
// which is the only on-device proof that CAN frames are being received.
// DEBUG にすると 10 秒ごとに TWAI・リングバッファ・BLE の件数が出る (CAN 受信の唯一の確認手段)。
#define LOG_LEVEL LOG_LEVEL_DEBUG

// GPIO pins connected to the CAN transceiver (SN65HVD230)
// RX_PIN : wired to the transceiver R (RXD / CRX) output
// TX_PIN : wired to the transceiver D (TXD / CTX). On these VP230 modules an open
//          D input floats dominant and R stops toggling, so D must be tied high.
//          In TWAI_MODE_LISTEN_ONLY the controller never drives TX low, so the
//          bus is still never driven.
// RX_PIN : トランシーバの R (RXD / CRX) へ配線
// TX_PIN : トランシーバの D (TXD / CTX) へ配線。このモジュールは D 開放だと R が出なくなるため H に固定する。
//          LISTEN_ONLY では TX が L になることは無いのでバスは駆動されない。
#define RX_PIN 4
#define TX_PIN 5

// Serial baud rate
#define SERIAL_BAUD_RATE 115200

// TWAI RX buffer size
// Reduce to 64 if "Failed to create ring buffer." appears at boot
// 起動時に "Failed to create ring buffer." が出る場合は 64 に減らす
#define TWAI_RX_QUEUE_LENGTH 128

// Bluetooth 5.0 support, 1 to enable, 0 to disable
// The classic ESP32 is BT 4.2 and has no extended advertising: keep this commented out
// 無印 ESP32 は BT 4.2 のため有効化不可。コメントアウトのままとする
// #define CONFIG_BT_NIMBLE_EXT_ADV 1

// Limit the rate of messages sent to RaceChrono to x CAN messages each second
// Can be further refined per PID through the list below
// Start low and raise only the channels that matter in getUpdateRateHz()
// まず低めにし、必要なチャンネルだけ getUpdateRateHz() で引き上げる
#define DEFAULT_UPDATE_RATE_HZ 2

// An optional list of PIDs and their associated rate limit.
// Limits the update rate of the specified PID to the returned value.
// All IDs are 29-bit and were confirmed on the Abarth 595 Series 4 with the
// tools/can_monitor sketch (2026-09-07 in-car logs, build manual chapter 3).
// すべて 29 ビット ID。tools/can_monitor で実車確認済み (2026-09-07、手順書 3 章)。
uint8_t getUpdateRateHz(uint32_t can_id)
{

```

```

switch (can_id)
{

case 0x0618A001: // RPM b2-3 BE, accelerator pedal b7 (0-255 = 0-100 %) / 回転数・アクセル
    return 20;
case 0x0030A002: // Steering angle 20-bit (b3b4b5 >> 4), 0.1 deg, Left + / 舵角
    return 20;
case 0x0210A006: // Vehicle speed b4-5 BE / 128 = km/h / 車速
    return 10;
case 0x0218A006: // Wheel speeds 4 x 16-bit BE / 16 = km/h / 4 輪速
    return 10;
case 0x0810A000: // Brake switch b2 upper nibble (1 off, 7 on) / ブレーキ SW
    return 10;
case 0x0010A006: // Brake pressure b1-2 BE (0 ... ~0x11B) / ブレーキ圧
    return 10;
case 0x0628A001: // Clutch b5 bit5 (0x20) / クラッチ
    return 5;

default:
    return DEFAULT_UPDATE_RATE_HZ;
}
}

#endif // CONFIG_H

```

7-5. Arduino IDE のボード設定

リポジトリの README は ESP32-S3 を前提としているため、無印 ESP32 では以下のように読み替えます。「ESP32 Dev Module」の既定値と異なるのは Upload Speed のみです。

README の記載	無印ESP32での設定
Board	ESP32 Dev Module
USB CDC On Boot	項目なし → 無視
Upload Mode: USB-OTG CDC (TinyUSB)	項目なし → 無視 (通常の UART 書き込み)
USB Mode: Hardware CDC and JTAG	項目なし → 無視
Events Run On: Core 1	Core 1 (既定値。作者は必須と明記)
Arduino Runs On: Core 1	Core 1 (既定値。作者は必須と明記)
Partition Scheme: Default	Default 4MB with spiffs (1.2MB APP) のまま。ビルド結果 604,612 bytes (46%) で収まる
Upload Speed	115200 (921600 でも書けるが、失敗時は 115200 に落とす)

7-6. 書き込みと起動確認

1. 装置を車両 (OBD) に繋いでいない状態で、USB ケーブルで PC と接続する。
2. ポートを選択し、スケッチを書き込む。
3. シリアルモニタ (115200bps) を開き、起動ログに `Driver started.` と `BLE started.` が出ることを確認する。

本番ファームウェアの件数ログは「受信 0 件のときは出ない」

LOG_LEVEL が DEBUG のとき、10 秒ごとに `N messages received from TWAI in the last 10 seconds.` が出ますが、この行は `twai_receive()` が 1 件でも成功した後にしか実行されない構造です。机上や受信できていない車内では何も出ないのが正常で、「ログが出ない」＝「受信 0 件」です。受信の有無は本番ファームではなく `tools/can_monitor`（第10章）で確認してください。

USB 給電と車両 12V の同時接続について

Freenove ボードは回路図が公開されておらず、USB の 5V と「5V」ピンの間に逆流防止ダイオードがあるかは不明です。本書では「USB 給電と 12V 給電を同時にしない」を原則とします。書き込み・シリアル確認はすべて USB 給電のみで行い、車内でのシリアル作業（第10章）はプラグ 16 → MP1584EN の 12V 線を繋ぐ前に済ませます（トランシーバは Freenove の「3.3V」から給電されるため、USB 給電だけで CAN 受信も動作します）。12V 線を繋いだ後に USB を使うときは、装置を OBD から抜きます。

8. 性能に関する期待値

- 作者の README では、ESP32-S3 で 120Hz 級（合計では約 200Hz、Google Pixel 7 で計測）に達し、シングルコアの ESP32-C3 でも同等、BLE 5.0 の拡張アダプタイズを有効にしても差が無かったと記載されています。ボトルネックは BLE リンクの転送速度です。
- したがって BT 4.2 の無印 ESP32 でも S3 と同程度が見込めます（有力）。実効レートはスマートフォン側（iOS は接続間隔の下限が長め）に左右されるため、実測して決めます。
- 設定方針：DEFAULT_UPDATE_RATE_HZ は 2 のまま、必要な ID だけ `getUpdateRateHz()` で引き上げる（7-4 の値で合計約 85 フレーム/秒）。実車の C-CAN は約 1,000 フレーム/秒・27 ID で、すべてを通そうとすると BLE 側で詰まり全体のレートが落ちます。
- 実測方法：LOG_LEVEL は既定で DEBUG なので、受信中は 10 秒ごとに「`messages received from TWAI`」「`messages sent to ring buffer`」「`messages sent to RaceChronoBle`」が出ます。ring buffer 行と RaceChronoBle 行の差が広がる、または `Receive ring buffer overflow` が出るなら BLE が飽和しています。レートを下げてください。

9. ベンチテスト（車両接続前の必須確認）

以下をすべて満たすまで、車両には接続しないでください。判断はテスターの直流値ではなく、装置が表示する結果（LED・シリアル）で行います。

9-1. 通電前（テスター）

- ☐ SN65HVD230 基板の CANH-CANL 間が 120Ω 付近でない（120Ω 除去済み。数十 kΩ 以上または OL）
- ☐ MP1584EN の出力が 5.00V ± 0.05V で安定しており、半固定抵抗が固定されている
- ☐ プラグ 16 → MP1584EN IN+、プラグ 5（/4）→ IN- の極性が正しい（逆接続保護は無い）
- ☐ Freenove 「4」の線が SN65HVD230 の R（CRX）に、「5」の線が D（CTX）に入っている（入れ違えると受信できない）
- ☐ 12V ラインと GND ラインの間、および CANH-CANL 間にショートが無い
- ☐ すべての配線に熱収縮チューブまたはカプトンテープで絶縁処理がされている

9-2. USB 給電（12V 未接続）

- ☐ 本番ファームで起動ログが出る（Driver started. / BLE started.。件数ログは机上では出ない）
- ☐ スマートフォンで「RaceChrono DIY ESP32」が見える（iOS は設定 > Bluetooth の一覧、または BLE スキャナアプリ）
- ☐ RaceChrono Pro からデバイスを追加し、接続が確立する（シリアルに RaceChrono connected.）
- ☐ tools/can_monitor を書き込むと、HB 行の pin"4": level=H になる（机上ではフレームが無いので RX NONE と青 LED が正常。L なら D の配線か「4」の配線を疑う）

9-3. 外部給電（USB を外し、9V 乾電池を MP1584EN の IN+／IN- に当てる）

- ☐ 給電すると ESP32 が起動し、スマートフォンから BLE が見える・接続できる（USB は外しておく。プラグ 16 の線はまだ IN+ に繋がなくてよい）
- ☐ MP1584EN・ESP32 に発熱・異臭が無い。テスターで Freenove の「5V」ピンが 5.0V 付近
- ☐ 給電を外すと BLE が消える

9-4. トランシーバの機能テスト（任意・tools/twai_selftest）

受信部が本物の差動信号を復号できるかを机上で確かめる自己診断スケッチです。ユニットの配線に一時的にジャンパ線を足して使います：D（CTX）を「5」から外して「3.3V」へ、「25」→CANH、「26」→CANL（読み戻し用の「34」→CANH、「35」→CANL は任意）。書き込んでシリアルを見ると、RXTEST（ESP32 の DAC で CANH/CANL に 7 状態の電圧を加えて R の応答を見る）と TWAI-B（ESP32 が 500 kbps の差動信号を直接作って自分で受信する）が走り、VERDICT CAR-READY と緑 LED で合格です。TWAI-A（モジュールのドライバ経由）は終端抵抗を外した基板では FAIL になるのが正常なので無視します。終わったらジャンパを外し、D を「5」に戻してから本番ファームまたは can_monitor を書き込みます。

10. 車両接続と CAN ID の確認

受信確認と ID 特定には、リポジトリの tools/can_monitor/can_monitor.ino（Rev.4）を使います。本番ファームと同じ配線・同じ Listen Only 設定で動く受信モニタで、書き込んで挿すだけで受信状態を LED とシリアルに表示し、操作で変化した ID とバイトだけを行として出します。追加ライブラリは不要、ボード設定は本番と同じ「ESP32 Dev Module」です。

RGB LED「16」	意味
青	フレーム未受信（机上、IG OFF、配線不良）
緑	受信中・エラー無し
黄	受信中だがバスエラーあり（HB 行の bus_err が増え続けるなら配線を疑う）
赤	受信できていたのに 2 秒以上途絶（IG OFF、コネクタ抜け）

10-1. ログを Mac に保存する準備

Arduino IDE のシリアルモニタは画面が流れると消えるので、ターミナルの screen でファイルに残します。シリアルモニタは閉じておきます（ポートを占有するため）。

```
mkdir -p ~/Desktop/canlog
cd ~/Desktop/canlog
```

画面に流れつつ `~/Desktop/canlog/screenlog.0` に追記保存されます。終了は **Ctrl-A → K → y**。止まらないときは別のターミナルで `pkill -x screen`。screen を起動したディレクトリにログができるので、リポジトリの中で起動しないこと。

10-2. 初回接続（受信確認）

1. プラグ 16 → MP1584EN IN+ の 12V 線はまだ接続しない（USB 給電で作業する）。can_monitor を書き込み、OBD-II ポート（運転席足元・低い位置）に接続する。ケーブルは上方向へ逃がし、結束バンドで既存ハーネスへ固定する。
2. ノート PC の USB で給電し、10-1 の screen を起動する。
3. イグニッションを ON にする（エンジン始動でもよい。IG OFF では C-CAN が止まっていて 0 件が正常）。
4. 5 秒ごとの HB 行が RX OK、LED が緑になれば受信成功（実車では約 1,000 フレーム/秒・27 ID）。RX NONE のままなら HB の 2 行目 `pin"4": level=... edges/20ms=...` で切り分ける（下表）。
5. 受信できたら装置を OBD から抜き、プラグ 16 の線を MP1584EN IN+ に接続し、本番ファームを書き込んで、USB なしで OBD に挿し直し、RaceChrono Pro から接続できることを確認する。

HB 2 行目の表示	意味と次に見る所
<code>edges/20ms > 0</code>	信号は「4」まで来ているが TWAI が復号できない。ビットレート違い（ピン 1/9 の B-CAN に配線していないか）、CANH/CANL の入れ違い
<code>edges = 0、 level=H</code>	モジュールは動作しているがバスに信号が無い。IG ON か、OBD ピン 6/14 がモジュールの CANH/CANL まで来ているか（圧着・プラグの穴位置）
<code>edges = 0、 level=L</code>	R (CRX) が駆動されていない。D (CTX) が「5」に繋がっているか（開放だとこうなる）、モジュールの 3V3/GND、CRX → 「4」
<code>bus_err > 0</code> で 0 件	信号は届いているが復号できない。ビットレート／CANH-CANL 逆

※ HB 行の `alerts=0x1000` (ERR_PASS) は Listen Only の仕様で常に出ます。異常ではありません。

10-3. CAN ID の特定（can_monitor の BASELINE / EVENT）

can_monitor は最初のフレームから 10 秒間を BASELINE として各 ID・各バイトの値の範囲を覚え、その後は範囲を外れたバイトがあった瞬間だけ EVENT 行を出します。カウンタやチェックサムは BASELINE 中に全範囲を取るので出ず、ハンドルやペダルで動いたバイトだけが [XX] 付きで流れます。第3章の ID はこの方法で特定しました。再確認や追加の信号を探すときの手順：

1. 10-2 と同じく USB 給電・IG ON（エンジンをかけたままでも可）で挿し、BASELINE 開始 から 10 秒は何も触らない。終わると全 ID の一覧表が出る。
2. 操作の直前に screen の画面で単語を打って Enter する（例 `steer`、`brake`、`th50`、`drive`）。`MARK t=...s` `steer` がログに残り、直後の EVENT と対応づけられる。
3. 操作は 1 つずつ、間に 3 秒ほど空ける。値の対応を取るには「保持」が有効：ハンドルを左 90° で 3 秒保持→戻す→右 90° 保持→左右ロック保持、アクセルは IG ON・エンジン停止で 1/4・1/2・全開を各 3 秒保持（回転数が動かないのでペダル位置だけが分かる）、車速は 10～20 km/h の徐行。
4. 最後に 30 秒待って TABLE が 1 回出てから終了する。screenlog.0 を読み、MARK の直後に [XX] が出た ID とバイトを表にする。

補足：OBDLink EX（STN コマンド）でも確認できる

手持ちの OBDLink EX を screen で開き、ATZ ATE0 ATH1 ATS1 ATCAFO STP 32 STMA の順に送ると生 CAN（500 kbps、11/29 ビット）が流れます（29 ビット ID は 02 18 A0 06 … のように先頭 4 バイト）。STFPA 0618A001, 1FFFFFFF と STM で ID を絞れます。DIY 機とは独立した確認手段として残しますが、操作との対応づけは can_monitor の EVENT/MARK の方が確実です。

10-4. RaceChrono Pro 側の設定

1. 公式チュートリアルを経路は「Settings → Add other device → Add other device → RaceChrono DIY」（日本語 UI では「設定 → その他のデバイスを追加 → …」）。BLE スキャンの一覧から RaceChrono DIY ESP32 を選ぶ。
2. デバイス追加後、車両プロファイルに「CAN-Bus チャンネル」の項目が現れる。下表の PID（10 進）と式を 1 チャンネルずつ登録する。式の raw はフレームの 8 バイト、bytesToUint(raw, offset, length) はビッグエンディアンの符号なし整数、bitsToUint(raw, bitOffset, bitLength) はビット切り出し。

チャンネル	PID（10進）	式	備考
Engine RPM	102277121	bytesToUint(raw, 2, 2)	rpm
Throttle position (%)	102277121	bytesToUint(raw, 7, 1) / 2.55	ペダル 0～100 %
Steering angle	3186690	(bytesToUint(raw, 3, 3) / 16 - 7400) * 0.1	度、左が正。アプリの表示が逆なら * -0.1。中立 7400 は直進中の値で微調整
Speed	34643974	bytesToUint(raw, 4, 2) / 128 / 3.6	m/s。チャンネルの単位が km/h なら / 3.6 を外す
Wheel speed FL / FR / RL / RR（左前・右前・左後・右後）	35168262	bytesToUint(raw, 0, 2) / 16 / 3.6 (offset 0 / 2 / 4 / 6)	m/s。輪の順序は推定（片輪だけ段差を越えて確認）
Brake pressure	1089542	bytesToUint(raw, 1, 2)	生値（単位未確定）
Brake（スイッチ）	135307264	bitsToUint(raw, 16, 4) >= 3	0 / 1
Clutch	103325697	bitsToUint(raw, 42, 1)	0 / 1（b5 の bit5）

※ DIY BLE デバイスの追加と CAN-Bus チャンネル機能には RaceChrono Pro（有料版）が必要です。画面名や式の関数名はアプリのバージョンで変わるため、実機で確認したうえで本書を更新してください。

11. トラブルシューティング

症状	確認事項
ビルドエラー：'class NimBLEAdvertising' has no member named 'setScanResponse'	NimBLE-Arduino が 2.x になっている。ライブラリマネージャで 1.4.3 を選んで入れ直す（第7章）。

ビルドエラー：RX_PIN 等が未定義	config.h がスケッチと同じフォルダにあるか確認。マクロ名は CAN_RX_PIN ではなく RX_PIN。
ビルドエラー：RaceChrono.h が見つからない	arduino-RaceChrono ライブラリを libraries フォルダへ手動配置したか確認（7-2）。
ビルドエラー：driver/twai.h や esp_mac.h が見つからない	ESP32 ボードパッケージが古い、または壊れている。esp32 3.x 系を入れ直す。
起動時に Starting BLE. の直後に ESP_ERR_INVALID_STATE (esp_bt_controller_init) で再起動を繰り返す	arduino-esp32 3.3.x が BLE 用メモリを解放してしまう。.ino の #include <esp32-hal-alloc-ble-mem.h> が残っているか確認（7-3）。
起動時に Failed to create ring buffer.	RAM 不足。.ino 内の xRingbufferCreate(65536, ...) を 16384 に、TWAI_RX_QUEUE_LENGTH を 64 に減らす。
起動時に Driver failed to start.	RX_PIN / TX_PIN の GPIO 番号を確認。
本番ファームで件数ログが 1 行も出ない	受信 0 件（ログの構造上、0 件だと表示されない）。can_monitor で HB 行を見る（10-2）。
can_monitor で pin"4": level=L 固定	R (CRX) が出ていない。最も多い原因は D (CTX) の開放。D が「5」（または「3.3V」）に繋がっているか、モジュールの 3V3/GND、CRX → 「4」の順に確認。「4」に直接 3.3V を当てて H になれば ESP32 側は正常。
level=H で edges=0、IG ON でも 0 件	バスの信号がモジュールに届いていない。プラグの穴位置（図 5-2 の③で数える）、6=CANH/14=CANL の圧着、CANH/CANL の線がモジュールまで導通しているか。
edges>0なのに0件、bus_errが増える	復号できていない。ピン 1/9（B-CAN 50 kbps）に繋いでいないか、CANH/CANL が逆でないか。
載せ替えた別のモジュールでは受信するが、元のモジュールは D を H にしても R が 0V	そのモジュールの受信部が動いていない。原因の特定はせず、動く方を使う（本書の製作でも 1 枚がこの状態になった）。
受信数は増えるが RaceChrono に値が出ない	PID・式の誤り（10-4 の表と照合）。DEBUG ログの Allow PID ... でアプリからの要求が届いているか確認。
データレートが遅い・途切れる	BLE の帯域上限。第 8 章の指標で飽和を確認し、getUpdateRateHz() のレートを下げる。
twai_selftest の TWAI-A が FAIL	終端抵抗を外した基板ではモジュール経由のループは通らない（正常）。判定は RXTEST と TWAI-B で行う（9-4）。
テスターでトランシーバの Vref（5 番ピン）が 0V	この基板では正常。故障の根拠にしない。判定は機能テスト（9-4、10-2）で行う。
ESP32 が起動しない・発熱する	直ちに電源を切る。MP1584EN の出力電圧を再測定する。
放置後にバッテリーが上がる	装置を挿したまま放置した。OBD-II ピン 16 は常時通電なので、使わないときは抜く。

12. 運用上の注意

- 使わないときは装置を OBD ポートから抜く。本装置は挿している間、12V 側で数十 mA を消費し続けます（目安として 1 日あたり 1Ah 前後）。挿したまま放置すると 1~2 週間でバッテリーに影響が出ると見込んでください。
- 夏季の車内は高温になります。長期駐車時は本体を取り外すことを推奨します。
- OBD ポート周辺は乗降時に膝が当たる位置です。本体とケーブルが運転操作を妨げないように固定してください。
- 走行中に操作が必要な状態にしないでください。設定変更は必ず停車時に行ってください。
- 本装置は受信専用ですが、車両の電気系統に接続する改造行為です。実施は自己責任で行ってください。
- 将来の改善案：CAN の無音（車両スリープ）を検出して deep sleep に入り、「4」の信号で復帰する処理を入れると、挿しっぱなしでも消費を抑えられます。

参考リンク

- ファームウェア（自分のフォーク）：github.com/katsuhaa/racechrono-diy-esp32 / 上流：github.com/kevenduchesneau/racechrono-diy-esp32
- ライブラリ：github.com/timurrrr/arduino-RaceChrono
- BLE DIY プロトコル仕様：github.com/aollin/racechrono-ble-diy-device
- CAN ID 事例集（他車種の参考）：github.com/timurrrr/RaceChronoDiyBleDevice
- RaceChrono DIY チュートリアル：racechrono.com/article/2572 / DIY フォーラム：racechrono.com/forum/categories/diy-builds
- NimBLE-Arduino 1.x→2.x 移行ガイド：github.com/h2zero/NimBLE-Arduino/blob/master/docs/1.x_to2.x_migration_guide.md
- ESP-IDF TWAI ドキュメント / ESP32 エラッタ：docs.espressif.com
- Freenove ESP32-WROOM ボード（ピン配置図・CH340 ドライバ）：github.com/Freenove/Freenove_ESP32_WROOM_Board
- Fiat/Alfa の OBD ピン割当（C-CAN 6/14、B-CAN 1/9）：alfaobd.com/androidhelp.html
- OBDLink Family Reference and Programming Manual（STP / STM / STMA / STFPA）：scantool.net
- Fiat 500（2010）CAN 解析ノート：github.com/P1kachu/talking-with-cars/blob/master/notes/fiat-500.txt / Fiat 500L CAN 復号（FiatMon）：github.com/VeryBusyBee/FiatMon

付録 A. can_monitor の出力の読み方

行	内容
HB t=...s WATCH fps=... total=... ids=... bus=... rx_err=... bus_err=... missed=... qfull=... alerts=...	5 秒ごとの状態。fps は受信レート、ids は見えている ID 数、missed は取りこぼし（0 が正常）、alerts=0x1000 は常に出る。続く行に RX OK か RX NONE、および候補 ID の仮デコード（RPM・車速・輪速・ブレーキ）。
==== BASELINE DONE - ID inventory ==== / ==== TABLE ====	全 ID の一覧。16 進と 10 進の ID、11/29 ビット、レート（Hz）、長さ、最新データ、ノイズバイト数。データの大文字は BASELINE 中に変化しなかったバイト、小文字は変化したバイト。

EVENT t=…s 0030A002 (3186690) B1 A3 FF 01[E1][91] [4E][50]	角括弧のバイトが BASELINE の範囲を外れて変化した。同じ ID は最短 0.25 秒間隔。
NEWID t=…s …	BASELINE 後に初めて現れた ID。
MARK t=…s steer	screen で打った文字列のエコー。操作の目印。
pin"4": level=H edges/20ms=0 "34"(CANH)=… mV "35" (CANL)=… mV	受信 0 件のときだけ出る切り分け行 (10-2 の表)。「34」「35」は未配線なら無視。

改訂履歴

版	日付	内容
Rev. 1.0	2026-09-04	初版
Rev. 1.1	2026-09-05	7-2：NimBLE-Arduino を 1.4.3 に固定 (2.x では arduino-RaceChrono がビルド不能。実際のエラーを 11 章に記載)。arduino-RaceChrono の導入コマンドを追記。7-1：「esp_mac.h のため 2.x 系ではビルド不可」を削除 (esp_mac.h は 2.x 系にも存在)。3.3.11 でのビルド確認結果を追記。7-6／9 章／10-1：USB 給電と車両 12V の扱いを整理。書き込み・シリアル確認は「電源スイッチ OFF・USB 給電のみ」に統一。ベンチテストを通電前／USB／12V の 3 段階に再構成。10-2：RaceChrono 側で全 PID をログする手順を削除し、付録 A のスニファによる手順に置換。10-3：公式チュートリアルを経路に修正。裏付けの無い「デバイス種別の設定」を削除し、実機確認を明記。5 章コラム：「既知のエラッタ」を公式資料の記述に合わせて修正 (設計は変更なし)。8 章：無印 ESP32 の性能期待値を「S3 と同程度・実測で決める」に修正し、飽和の判定方法を追記。4-1／9 章：終端抵抗除去の判定を「OL」から「120Ω 付近でない」に修正。4-3：40 ピンに修正。3 章：2 台構成の未確認事項 (B-CAN の物理層、複数 DIY デバイス) を追記。12 章：バッテリー消費の目安を修正。部品表は変更なし。
Rev. 1.2	2026-09-05	部品表からヒューズ・逆接続保護ダイオード (1N5819)・TVS (SMBJ26A) を削除。5-1 の電源系統と 9-1 のチェックリストを追従。それ以外の章は変更なし。
Rev. 1.3	2026-09-05	10kΩ 抵抗を削除し、トランシーバの D (TXD) は 3V3 へ直結に変更 (データシート上 D は High または開放でレセシブ。抵抗は不要)。5-2・5 章コラム・9-1・config.h のコメントを追従。
Rev. 1.4	2026-09-05	D (TXD) を「開放 (無接続)」に変更。配線工程を減らして作業を確実にするため。D ピンの絶縁と D-GND 非導通のチェックを追加。3V3 へのジャンパ線は任意扱い。
Rev. 1.5	2026-09-05	3 章：Fiat 500 (312) 系の公開情報から既知 CAN ID 候補表を追加 (車輪速・車速・回転数・ブレーキ・状態ビット)。放送フレームは 29 ビット ID である旨に修正 (旧版の「11 ビット」は誤り)。AiM／Racelogic の資料により舵角は CAN-C 側にある見込みと記載し、2 台構成は予備案に格下げ。10-2 に「既知候補の確認」を追加、診断フレームの補足を修正。付録 A：スニファの ID 表示を 29 ビットは 8 桁に変更。

Rev. 1.6	2026-09-05	電源スイッチと外付け電解コンデンサ（100 μ F $\times$ 2）を部品表と結線から削除（スイッチは使わないときに OBD から抜けば足りる。コンデンサは DC-DC モジュール搭載分で代替、リセットが出る場合のみ 5V 側に追加）。USB と 12V の同時給電を避ける手順を「CAN ID 特定が終わるまで 12V 線を繋がない」に変更し、7-6・9 章・10-1・12 章を追従。6 章：出力調整は 7 $\sim$ 28V の任意の DC 電源（9V 乾電池など）で自宅のできる旨と電源の選択肢を追記。10-2：OBDLink EX を STN コマンドでスニファとして使う代替手段を追加。
Rev. 1.7	2026-09-06	5 章：配線の記述を各基板のシルク表記に統一（Freenove は「4」「5」「3.3V」「5V」「GND」）。Freenove ボードのピン配置図（図 5-1）・J1962 ピン配置図（図 5-2）・配線図（図 5-3）を追加。配線表を 1 本ずつの 9 本に書き直し、旧版で抜けていた MP1584EN OUT- $\rightarrow$ GND を追記。GND はプラグ 5 の 1 本で足りる旨と、端子を入れるプラグ位置を明記。7-3／7-4：ソースを自分のフォークのブランチ abarth595-esp32-wroom に変更し、config.h をリポジトリの内容と同期。7-5：ビルドサイズを 604,492 bytes に更新。10-1：DEBUG への切替手順を削除し、12V 線の接続を受信確認の直後に変更。10-2：ESP32 ファームウェアのスニファ手順を削除し、OBDLink EX（STN コマンド）による確認を本線に変更。付録 A をファームウェア差分から PC 側のログ取得・比較スクリプト（任意）に差し替え。9-3：9V 電池は MP1584EN の IN+／IN- に当てる表現に修正。部品表は変更なし。
Rev. 1.8	2026-09-07	実車での受信確認と CAN ID の特定を反映。 D（TXD / CTX）の配線を「開放」から Freenove 「5」 への接続に変更 （手持ちの VP230 基板では D 開放だと R が 0V に張り付き 1 フレームも受信できなかった。初回の車両接続で発覚し、机上で切り分け）。5 章：配線表を 10 本に（⑩「5」 $\rightarrow$ D を追加）、図 5-1・5-3 とコラムを追従、9-1 のチェックを追従。3 章：候補表を実車で確定した ID 表に差し替え（回転数・アクセル・舵角・車速・4 輪速・ブレーキ SW・ブレーキ圧・クラッチ）。舵角は C-CAN 側で確定し 2 台構成の記述を削除。7-3：.ino の BLE メモリ保持の 1 行（esp32-hal-alloc-ble-mem.h. arduino-esp32 3.3.x での起動ループ対策）を記載。7-4：config.h を同期（TX_PIN のコメント、getUpdateRateHz に確定 ID）。7-5：604,612 bytes。7-6：本番ファームの件数ログは 0 件だと出ない旨を追記。9-4：tools/twai_selftest による機能テストを追加。10 章：受信確認と ID 特定を tools/can_monitor（LED 判定・HB・BASELINE/EVENT・MARK）と screen によるログ保存の手順に置換、OBDLink 法は補足に移動。10-4：RaceChrono のチャンネル設定表（PID 10 進と式）を追加。11 章：BLE 起動ループ、「4」 L 固定、Vref 0V、selftest の TWAI-A FAIL、載せ替え判定を追加。付録 A を can_monitor の出力の読み方に差し替え（旧 obdlink_canlog.py は削除）。部品表は変更なし（受信不能になった SN65HVD230 基板を予備に載せ替えた経緯を 2 章の補足に記載）。